

Best Practices for ADK Agents

A reusable checklist combining general agentic-system discipline with ADK-specific habits — clear tool contracts, structured state handling, traceability, and evaluation-first development.



What Makes an Agent Reliable?

1 — Reliability

A strong agent should do the right thing repeatedly, even under varied inputs and edge cases.

2 — Clarity

Explain itself well enough for debugging. Decisions should be traceable and understandable.

3 — Safety

Avoid unsafe actions. Guardrails should block, redact, or redirect based on clear policy.

4 — Traceability

Leave a visible trail of decisions so failures can be diagnosed from logs and traces.

1 — Effective Prompts

Keep instructions **specific, short, and operational**. Describe role, scope, boundaries, and decision rules in plain language.

What to Avoid

Avoid mixing policy, task logic, and examples into one dense block. Separation of concerns applies to prompts too — each section should have a single purpose.



Scope & Tool Design

3

Why Narrow Scope Matters

Narrow scope reduces confusion and improves answer quality. Agents perform better when each one has a **clear responsibility** instead of trying to solve every problem at once.

- ❗ Split the system when tasks differ enough to benefit from specialization. An orchestrator can route work to focused agents instead of forcing one model to do everything.

4

The Best Shape for a Tool

Tools should be **small, explicit, and easy to validate**. Keep inputs simple, outputs structured, and tool names descriptive so the agent can choose them correctly.

- Small surface area — one job per tool
- Explicit inputs with clear types
- Structured, predictable outputs
- Descriptive names the agent can reason about

Tool Contracts & Stateful Memory

5

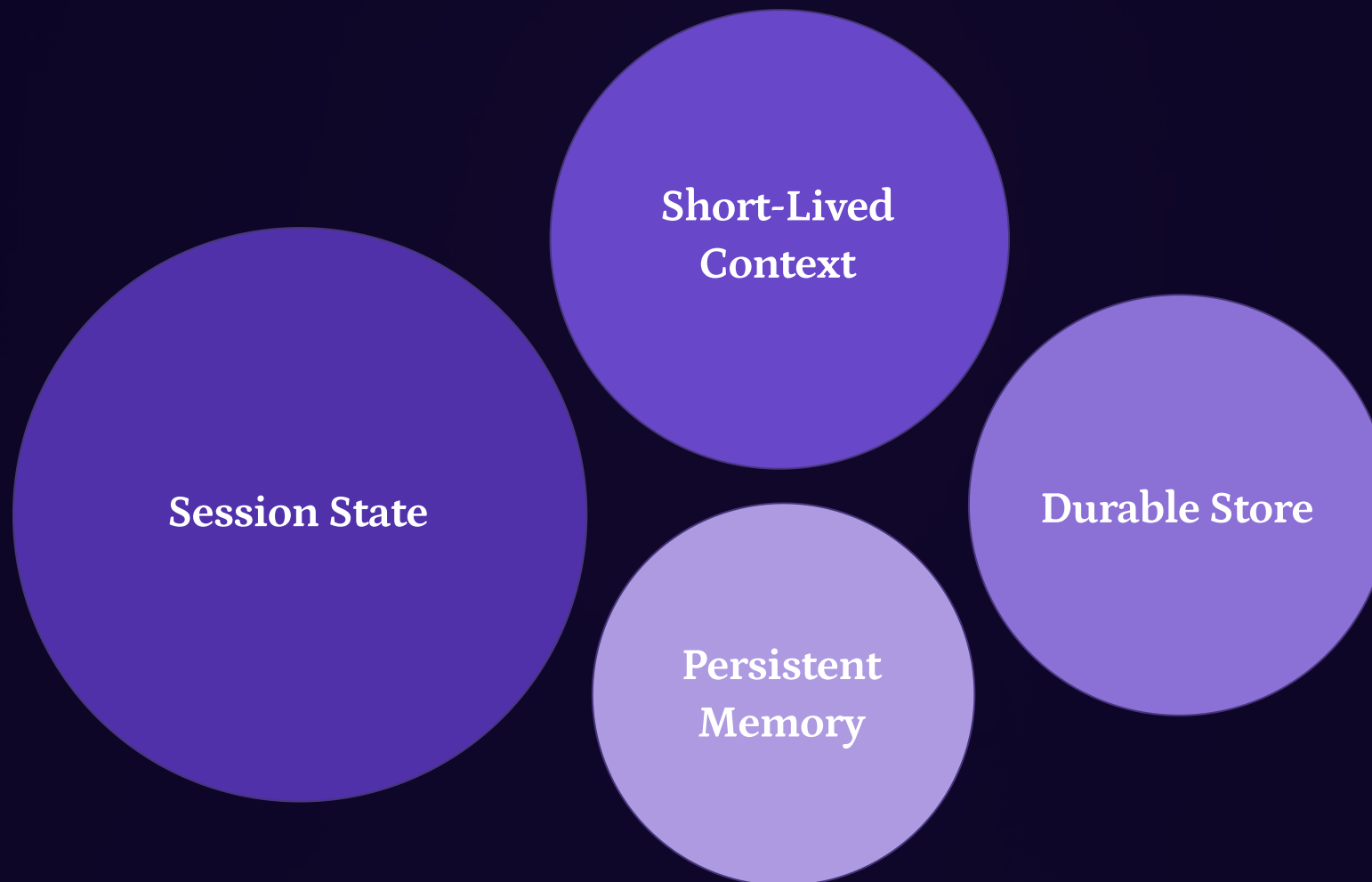
Designing Tool Contracts

Clear schema, predictable output, and strong input validation make tools easier for an agent to use. The agent should know what the tool needs, what it returns, and when it is safe to call it.

6

Handling Conversation Memory

Keep **short-lived context in session state** and persistent data in a durable store. Do not rely on prompts alone for memory that must survive across turns or restarts.



Separating transient and durable state keeps the system simpler, safer, and easier to scale — each layer has a distinct purpose and failure mode.



State Separation & Retrieval

7

Why Separate Transient and Durable State?

Transient state is for the **current conversation**, while durable state supports **recovery and reuse**. Splitting them keeps the system simpler, safer, and easier to scale.



Storing everything in one place creates coupling between short-lived session logic and long-term persistence — a recipe for hard-to-debug failures.

8

When Does Retrieval Improve an Agent?

Retrieval helps when answers must come from **current, domain-specific, or curated knowledge**. The agent should use retrieved context as evidence, not as decoration.

- Use retrieval for time-sensitive facts
- Use retrieval for proprietary knowledge bases
- Treat retrieved chunks as grounding, not filler

RAG Guardrails & Model Gateways



9 — Guarding RAG Flows

Ground answers in retrieved sources, detect weak or missing context, and allow the agent to say it does not know. **Good retrieval is about restraint as much as recall.**



10 — The Value of a Model Gateway

A gateway gives one interface for multiple models, providers, and fallback paths. That makes **routing, cost control, and resilience** easier to manage centrally.

⚠️ Never let weak retrieval silently produce confident-sounding answers. Build explicit detection for low-confidence or missing context and surface it to the caller.

Model Selection & External Integrations



Choosing the Right Model

Match model capability to task difficulty. Use **lighter models** for simple, frequent requests and **stronger models** for reasoning-heavy or high-risk tasks.

12

Safe External Integrations

Validate inputs, constrain actions, and handle failures gracefully. **Never let raw user text flow directly into destructive operations.**

- Validate all inputs before dispatch
- Constrain the action surface of each integration
- Handle failures with structured error responses
- Log every external call for traceability



Structured Interfaces & Multi-Agent Orchestration

1

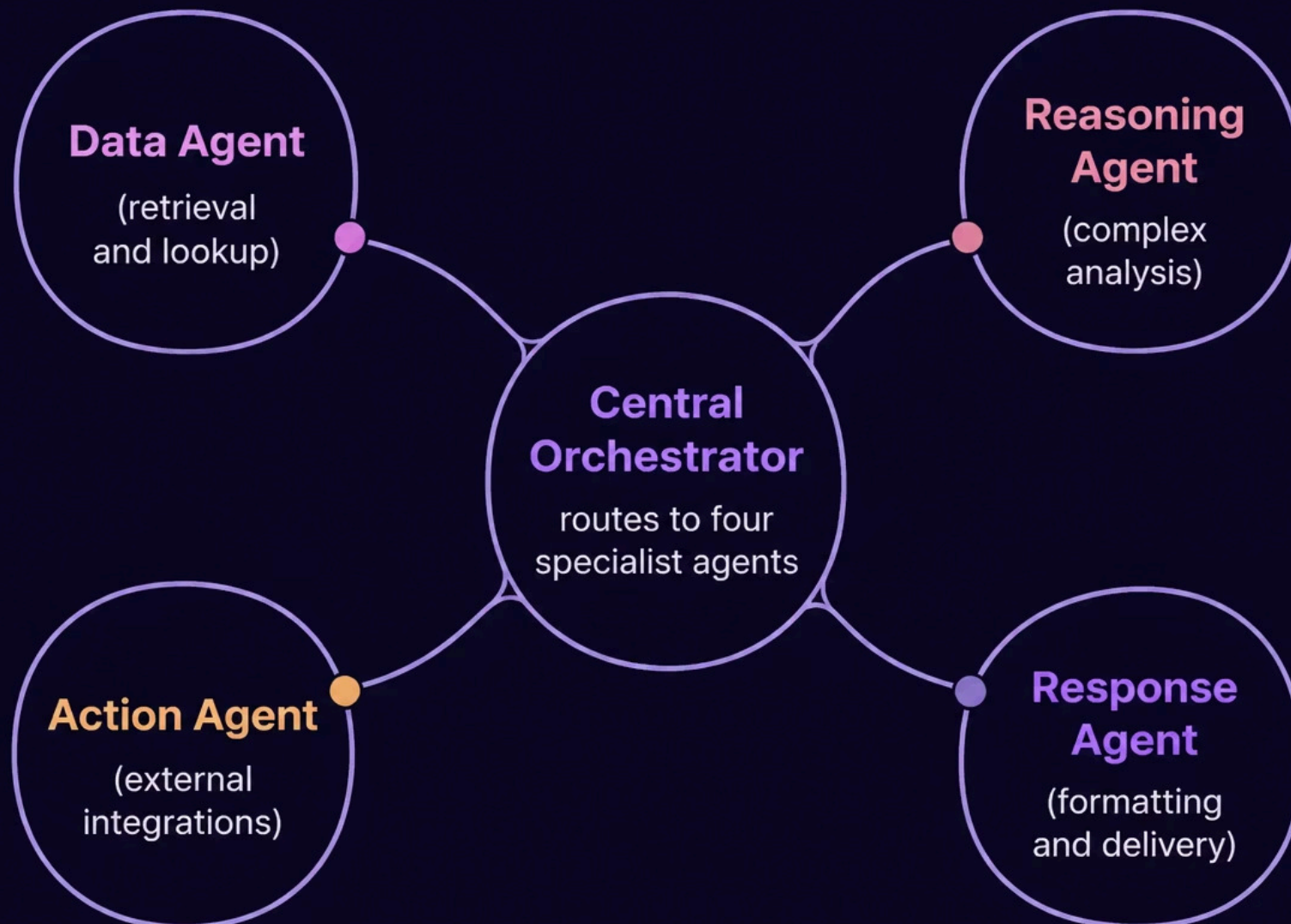
13 — Structured Interfaces

Stable contracts, typed inputs, and predictable outputs help an agent work well with external systems. When integrations are explicit, the agent can recover more easily from malformed responses.

2

14 — Orchestrating Multiple Agents

Split the system when tasks differ enough to benefit from specialization. An **orchestrator can route work to focused agents** instead of forcing one model to do everything.



Delegation & UI Generation

15

Making Multi-Agent Routing Effective

Use a **clear classification step**, **simple routing rules**, and **specialist agents** with distinct responsibilities. Keep delegation visible so failures are easy to trace.

- ❗ Opaque delegation is a debugging nightmare. Every routing decision should be logged with the reason it was made.

16

Why Agent Output Should Shape the Interface

A structured UI makes answers easier to trust and use. **Cards**, **badges**, and **state indicators** help users understand what the agent did and why.

- Cards for structured answer presentation
- Badges for confidence and source attribution
- State indicators for in-progress actions
- Error states that explain what went wrong



Voice Systems & Visible Reasoning



17 — Voice Pipeline Design

Latency, accuracy, and turn-taking behavior matter most. Keep speech flowing smoothly from **recognition** → **reasoning** → **synthesis**.



18 — Why Expose Reasoning?

Visible reasoning makes complex decisions easier to **inspect, debug, and improve**. It should remain concise and useful, not become a dump of private chain-of-thought.

- ❏ In voice-first agents, every added millisecond of latency degrades the conversational feel. Optimize the full pipeline — not just the model inference step.

Observability & Continuous Evaluation

19

What Tracing Should Tell You

Traces should show **routing, tool use, model choice, latency, and errors**. If you cannot explain a failure from logs and traces, the system is too opaque.

20

The Role of an Eval Suite

Evaluations catch regressions before users do. Keep a **repeatable set of scenarios** that reflects core tasks, edge cases, and safety boundaries.

- Core task coverage — happy paths
- Edge cases — unusual inputs and formats
- Safety boundaries — injection and PII tests
- Regression gates — block broken deploys



Safety & Production Architecture



Prompt Injection

Block adversarial inputs that attempt to override system instructions or hijack agent behavior.



PII Leakage

Redact or mask personally identifiable information before it reaches model context or external logs.



Unsafe Tool Use

Constrain destructive tool calls with explicit validation and require confirmation for irreversible actions.



Off-Topic Drift

Redirect agents that stray outside their defined scope using policy-based guardrails, not ad hoc judgment.

22

Production Deployment Design

Separate **compute, state, secrets, observability, and external services**. A production design should be scalable, secure, and recoverable.



Guardrails should block, redact, or redirect based on **clear policy** rather than ad hoc judgment. Undocumented guardrails are impossible to audit or improve.

Monitoring & Secrets Management

6

Key Signals

Error rate, latency, token usage, cost, blocked requests, and tool failures — all must be tracked.

0

Hardcoded Secrets

Zero tolerance. Never hardcode credentials in prompts, code, containers, or shared examples.

23

What to Monitor

- Error rate and failure classification
- Latency at each pipeline stage
- Token usage and cost per request
- Blocked requests and guardrail triggers
- Tool call failures and retry rates

24

Safe Secrets Handling

Store secrets in a **managed secret store** and inject them at runtime. Never hardcode credentials in prompts, code, containers, or shared examples.

- ⊗ A single leaked credential in a shared example or container image can compromise an entire production environment.



The Best Development Habit & Closing Checklist

01

Build Incrementally

Start with the smallest working agent and add capability layer by layer. Each addition should be independently testable.

02

Test Often

Run your eval suite on every change. Evaluations catch regressions before users do — make them part of the deploy gate.

03

Keep Changes Small and Observable

Large, opaque changes are the enemy of reliable agents. Every change should be traceable, reversible, and scoped.

04

Guard at Every Layer

Good agent development is iterative, measured, and guarded — from prompt design through deployment and monitoring.

These practices form a reusable checklist across the full ADK training. They combine general agentic-system discipline with ADK-specific habits: **clear tool contracts, structured state handling, traceability, and evaluation-first development.**